基于 YOLO11n 的桌面物体检测与 ROS2 通信实验

李书宇
中国海洋大学信息科学与工程学部计算机学院
2024 级计算机科学与技术（中外合作办学）
学号：24020036035
课程：机器人集成小组项目　指导教师：周小伟

摘　要

本实验采用 YOLO11n 完成鼠标、键盘、笔记本电脑、杯子和头戴式耳机五类桌面物体的检测，并通过 ROS2 发布检测结果。训练数据共 5000 张，标注目标框 8507 个，其中 1000 张为自主采集来源素材，4000 张取自 Open Images V7。数据按母图分组划分，以防止同源增强图像分散到不同集合。模型以 640 像素输入训练 12 轮，在 650 张测试图像上取得 0.8385 的 Precision、0.7697 的 Recall、0.8386 的 mAP@0.5 和 0.5735 的 mAP@0.5:0.95。另选取 20 张单目标图片进行测试，正确识别 19 张，正确率为 95%。Apple M4 MPS 环境下，固定视频 130 个计时帧的平均端到端速度为 60.409 FPS。程序支持图片、视频和摄像头输入，可保存检测记录及错误样本，并发布结构化检测结果与带框图像。

关键词：目标检测；YOLO11n；桌面物体；ROS2；边缘视觉；迁移学习

***Abstract*—This experiment applies YOLO11n to the detection of five desktop-object classes: mouse, keyboard, laptop, cup, and headphones. Detection results are subsequently published through ROS2. The dataset comprises 5,000 images with 8,507 bounding boxes; 1,000 images originate from self-collected photographs and 4,000 are drawn from Open Images V7. All derivatives of a parent photograph were assigned to the same split to prevent source overlap across training and evaluation data. After 12 epochs of fine-tuning at an input size of 640 pixels, the model achieved a precision of 0.8385, a recall of 0.7697, an mAP at IoU 0.5 of 0.8386, and an mAP over IoU 0.5–0.95 of 0.5735 on 650 test images. It correctly recognized 19 of 20 additional single-object samples. A 130-frame timed video test on Apple M4 MPS yielded an average end-to-end rate of 60.409 FPS. The resulting program processes images, videos, and camera streams, saves detections and error cases, and publishes structured results together with annotated images.**

***Index Terms*—object detection, YOLO11n, desktop objects, ROS2, edge vision, transfer learning**

I. 引言

桌面抓取和人机交互不仅需要判断物体类别，还需要获得目标在画面中的位置。鼠标、键盘和笔记本电脑常在较小范围内同时出现；笔记本打开时，整机与可见键盘又形成相互包含的目标。光照、遮挡和拍摄角度的变化进一步增加了检测难度。由此，识别程序需要连续处理摄像头画面，并同步输出类别、边界框和置信度。

YOLO 在一次前向传播中同时完成类别预测和边界框回归，属于单阶段检测方法 [1]。SSD 则利用不同尺度的特征图处理尺寸差异较大的目标 [2]。这类方法省去了候选区域生成步骤，推理链路较短，更适合接入摄像头程序。本实验据此选用参数量较小的 YOLO11n，在五类桌面物体数据上继续训练，并通过 ROS2 发布检测结果。

实验包括数据整理、模型训练、结果测试和 ROS2 节点实现。除测试集逐类指标外，还按照课程验收口径测试 20 张单目标图片，并在固定视频上记录完整处理速度。实验未改动 YOLO11n 的网络结构，主要工作集中在数据构建、推理实现、结果留存和通信接口的衔接。

II. 相关工作

A. 单阶段目标检测

原始 YOLO 直接从整幅图像预测边界框及类别概率，省去了区域提议与二次分类 [1]。SSD 在多层特征图上设置默认框，使不同尺寸的目标能够在相应分辨率上完成检测 [2]。针对稠密检测中前景、背景样本不平衡的问题，Focal Loss 降低易分类样本的损失权重，使训练更关注困难样本 [3]。上述方法奠定了实时单阶段目标检测的基本技术路线。

数据增强在训练阶段改变样本外观，不会增加部署时的网络计算量。YOLOv4 将颜色变化、几何变换和 Mosaic 等方法组合用于检测训练，并比较了实时场景下的速度与精度 [4]。桌面物体通常不存在大幅形变，故本实验将旋转限制在较小范围，并配合平移、缩放、水平翻转及概率为 0.5 的 Mosaic。

YOLO11 由 Ultralytics 软件包提供、训练、验证、预测和模型导出使用统一接口，其中 n 规模模型面向轻量推理 [5]。鉴于 YOLO11 尚无独立的架构论文，本文以官方代码仓库标明具体模型版本；早期 YOLO 文献仅用于说明单阶段检测的基本思路。

B. 数据资源、评价方法与通信框架

Open Images 收录了自然场景图像及多种标注，其目标检测子集包含大量多目标画面 [6]。实验从 V7 版本筛选与任务对应的五类图片，并将边界框转换为 YOLO 格式。模型评价采用 Precision、Recall、mAP@0.5 和 mAP@0.5:0.95。其中 mAP@0.5:0.95 沿用 COCO 的多 IoU 阈值计算方式 [7]，各项数值由 Ultralytics 验证接口给出 [8]。

ROS2 通过主题传递传感器数据和机器人状态。发布者与订阅者约定主题名称和消息类型后即可交换数据，无需直接调用对方程序 [9]。目标检测因而可以作为独立节点运行，其余程序按需订阅数值结果或可视化图像。

III. 系统设计与方法

A. 总体架构

图1给出了程序流程。离线部分完成公开数据筛选、标签转换、数据划分和模型训练；在线部分读取图片、视频或

TABLE I
数据集划分与目标框统计

划分	图像	标签	空标签	目标框
Train	4000	4000	0	6773
Validation	350	350	0	627
Test	650	650	0	1107
合计	5000	5000	0	8507

USB 摄像头画面，输出逐帧日志、带框结果与错误样本。ROS2 节点沿用相同的推理配置并封装检测消息。训练入口与推理入口彼此独立，更换输入源不会影响模型训练代码。

每条检测记录包含类别编号、类别名称、置信度以及像素坐标边界框 (x_1, y_1, x_2, y_2)。图像缩放至 640×640 后输入网络，候选框经置信度筛选和非极大值抑制得到最终结果。结果同时用于绘制检测框和写入 CSV，带框图片或视频可按运行参数保存。

B. 模型训练策略

YOLO11n 由已有的五类模型权重初始化，并在重新整理的数据上继续微调。正式训练共 12 轮，批量大小为 32，输入尺寸为 640，随机种子为 0，同时开启确定性选项。优化器由 Ultralytics 自动选择，初始学习率参数、动量和权重衰减依次设为 0.01、0.937 和 0.0005。

训练阶段在线施加颜色与几何增强，其中色相、饱和度和明度参数分别为 0.015、0.7 和 0.4，旋转范围为 5 度，平移与缩放比例为 0.1 和 0.3。水平翻转和 Mosaic 的概率均为 0.5，MixUp 关闭。训练结束后按验证集指标保存最佳权重，并保留最后一轮权重。

IV. 数据集构建

A. 类别体系与标注

五个类别依次为鼠标（mouse）、键盘（keyboard）、笔记本电脑（laptop）、杯子（cup）和头戴式耳机（headphones），类别编号为 0 至 4。标签采用 YOLO 水平边界框格式，每行记录类别编号、归一化中心坐标及归一化宽高。对于打开的笔记本电脑，整机标注为 laptop，可见键盘区域另行标注为 keyboard；目标受到遮挡时，仅框选可见部分。

数据划分见表I。5000 张图像均有对应标签文件，共标注 8507 个目标框。训练集含 4000 张图像，验证集含 350 张，余下 650 张作为测试集。最佳权重按验证结果选取，测试集不参与训练过程。

B. 数据来源与划分控制

5000 张图片中，1000 张属于自主采集来源素材，另外 4000 张来自 Open Images V7。自主采集部分以 10 张独立拍摄的母图为基础：每张母图先统一尺寸，再通过颜色、亮度、尺度、平移或轻微旋转生成 99 个版本，与母图共同构成 100 张素材。因此，1000 张的统计口径包含实拍母图及其衍生图像，而非 1000 次独立拍摄。母图覆盖 mouse、keyboard 和 laptop，cup 与 headphones 由公开数据补充。公开图片保留原始来源 ID 与划分信息。

同一母图的不同增强版本若分别进入训练集和测试集，会造成同源信息泄漏并抬高测试结果。为此，10 张母图按 8:1:1 分配，同一母图生成的全部版本只进入一个集合。数据生成后依次检查图片可读性、图像与标签配对、类别编号及坐标有效性，并核对重复来源和跨集合母图。图2展示了部分自主采集来源素材及其标注，设定的图像变换未改变类别与边界框的对应关系。

TABLE II
五类目标框分布

类别	目标框数	比例（%）
mouse	1724	20.27
keyboard	1966	23.11
laptop	2549	29.96
cup	1397	16.42
headphones	871	10.24
合计	8507	100.00

各类目标框的统计结果见表II。laptop 共 2549 个，占 29.96%，而 headphones 为 871 个，占 10.24%。类别数量并不均衡，因此总体均值之外还需考察各类 Precision、Recall 和 AP。

V. 实验设置与评价方法

A. 实验环境

训练与软件测试环境为 macOS，处理器为 Apple M4，计算后端采用 MPS。Python、PyTorch 和 Ultralytics 版本分别为 3.12.4、2.13.0 和 8.4.133，训练与测试的输入尺寸均为 640。第 12 轮权重在验证集上的 Precision、Recall、mAP@0.5 和 mAP@0.5:0.95 分别为 0.7407、0.7512、0.7334 和 0.4621，据此选作后续测试模型。

B. 评价指标与测试协议

设真正例、假正例和假负例分别为 TP、FP 和 FN，则 Precision 与 Recall 定义为

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}. \tag{1}$$

预测框与真实框的交并比为

$$\mathrm{IoU} = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}, \tag{2}$$

其中 B_p 和 B_g 分别为预测框与真实框。mAP@0.5 在 IoU 阈值 0.5 下计算各类别 AP 的均值，mAP@0.5:0.95 则在 0.5 至 0.95、步长 0.05 的多个阈值上平均，因而对边界框定位更严格 [7], [8]。

测试分为三部分。第一部分在 650 张测试图像上统计总体与逐类指标，并生成 PR 曲线和混淆矩阵。第二部分从已保存的单目标图片中每类选取 4 张，共 20 张；当指定类别被检出时，该图计为识别正确。第三部分在同一段固定视频上测量速度，先运行 20 帧完成预热，再对后续 130 帧计时。计时覆盖读取、预处理、推理、后处理和绘制全过程，不局限于网络前向计算。

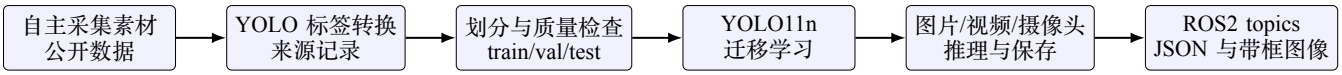


Fig. 1. 桌面目标检测系统的端到端处理流程。

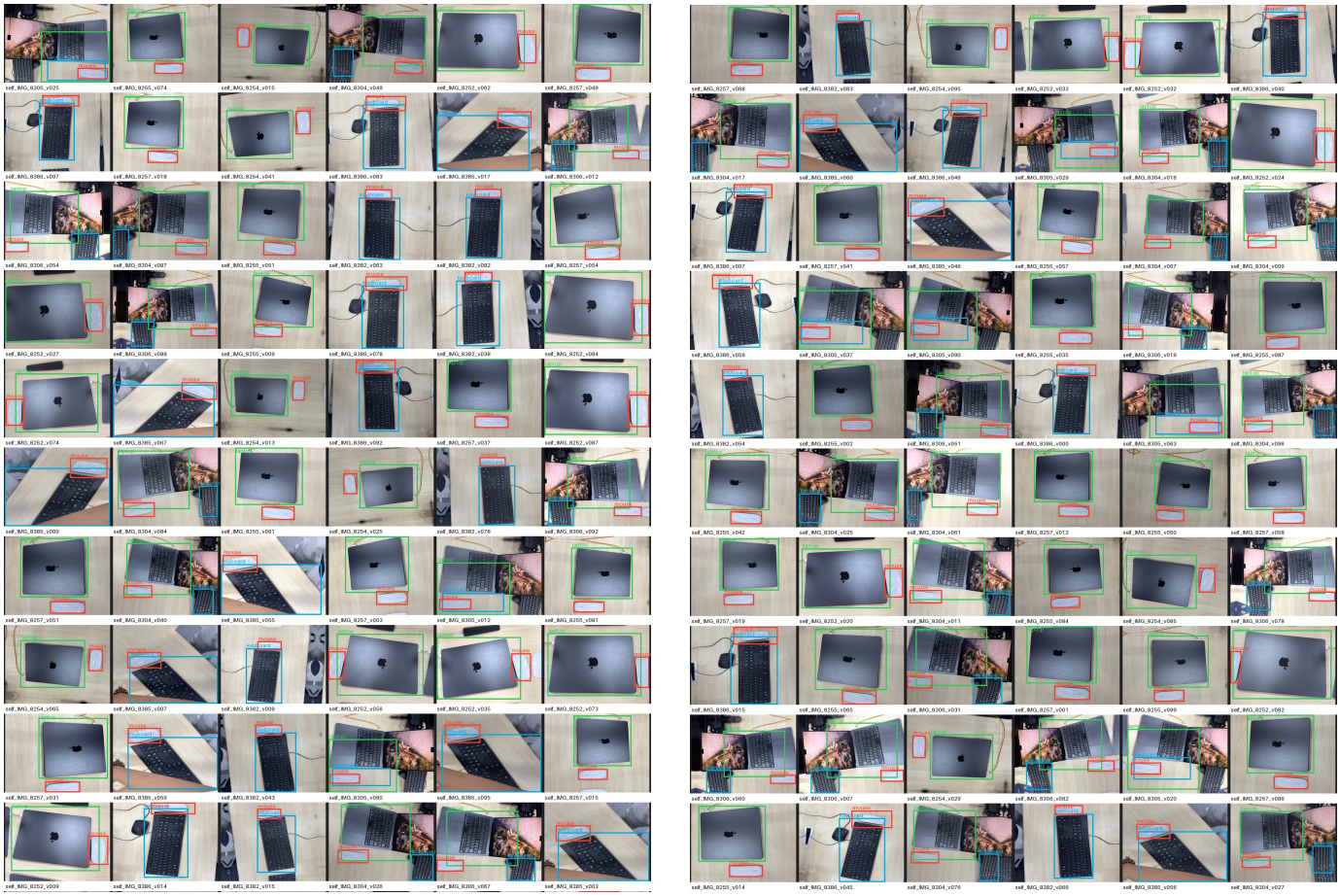


Fig. 2. 自主采集来源素材的增强与标注样例。

VI. 实验结果与分析

A. 测试集总体结果

650 张测试图像的检测结果列于表III。总体 Precision 为 0.8385，Recall 为 0.7697，表明漏检相较误检更为突出。mAP@0.5 达到 0.8386，而采用 0.5 至 0.95 多阈值平均后，mAP@0.5:0.95 为 0.5735。两项指标相差 0.2651，反映出模型在较宽松 IoU 条件下能够识别多数目标，但边界框在严格定位标准下仍有明显损失。

mouse 的表现最好，mAP@0.5 和 Recall 分别达到 0.9501 和 0.9101。keyboard 的 mAP@0.5 为 0.9012，说明将笔记本可见键盘与独立键盘归为同一类别后，模型仍能较好地学习其共同特征。laptop 的 Precision 达到 0.9121，但 Recall 仅为 0.7529，mAP@0.5:0.95 进一步降至 0.4599。检查保存的检测图发现，笔记本受到遮挡、开合角度变化较大或靠近画面边缘时，整机框较易发生漏检或偏移。

headphones 的 Precision、Recall 和 mAP@0.5 分别为 0.6041、0.5978 和 0.6203，均为五类最低。该类标注框数量最少，样本还包含展开、折叠及局部遮挡等差异较大的形态。现有数据只能表明样本较少与指标较低同时出现，不能据此认定二者具有直接因果关系。不同姿态、尺度和遮挡程度是该类数据中需要重点覆盖的变化因素。

TABLE III
测试集总体与逐类检测指标

类别	Precision	Recall	mAP@0.5	mAP@0.5:0.95
mouse	0.9325	0.9101	0.9501	0.7546
keyboard	0.8346	0.8469	0.9012	0.6361
laptop	0.9121	0.7529	0.8640	0.4599
cup	0.9091	0.7409	0.8571	0.6751
headphones	0.6041	0.5978	0.6203	0.3419
总体	0.8385	0.7697	0.8386	0.5735

B. 训练过程与 20 样本测试

图3记录了 12 轮训练过程。训练损失总体下降，验证集 mAP 在后几轮仍有提升，最佳权重出现在第 12 轮。验证损失虽有波动，但验证指标未出现连续下降。考虑到初始

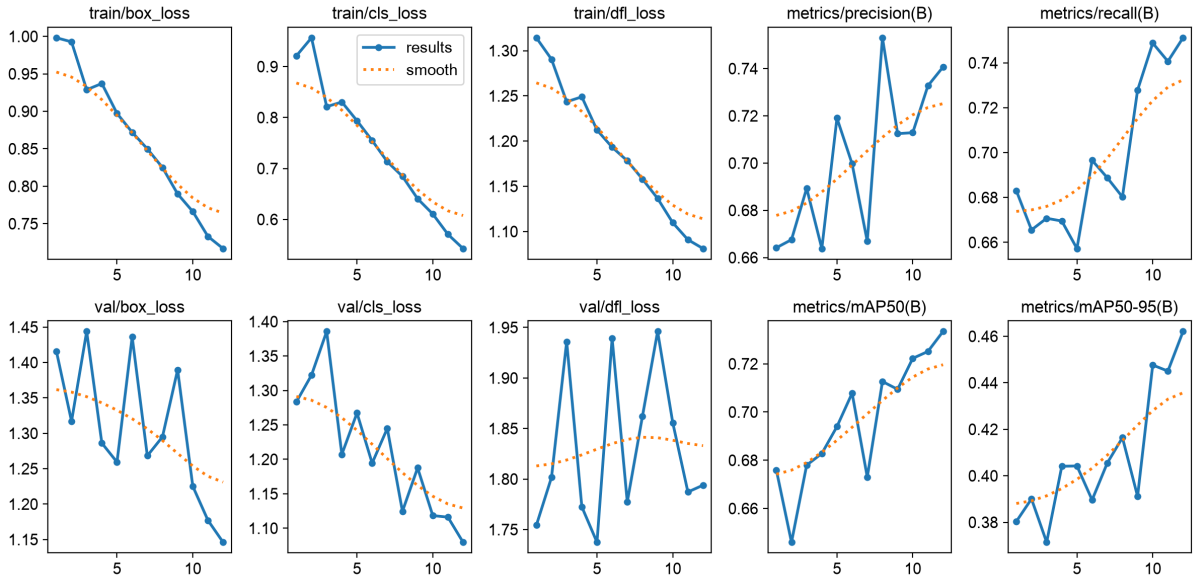


Fig. 3. YOLO11n 在 12 轮迁移学习中的损失和验证指标曲线。



Fig. 4. 650 张测试图像上的归一化混淆矩阵。

权重已在相同五类目标上训练，本实验采用较短周期继续微调。

20 张样本中有 19 张识别正确，正确率为 95%，高于实验要求的 80%。唯一的错误样本属于 laptop，模型未输出达到置信度阈值的 laptop 检测框。这与测试集上 laptop 的高 Precision、低 Recall 特征一致，主要错误形式为未生成有效检测框。该样本已单独保存，可结合背景、目标尺度和遮挡情况进一步分析。

图4给出了 650 张测试图像上的归一化混淆矩阵，用于观察各类别的正确识别、相互混淆和背景漏检情况。

C. 连续视频性能

固定视频共记录 130 个计时帧，模型推理平均耗时 10.087 ms。计入视频读取与结果绘制后，平均单帧耗时为 16.554 ms，对应 60.409 FPS。最低瞬时速度为 34.733 FPS，最大单帧推理耗时为 13.450 ms。由平均耗时可见，网络前向之外的处理约占 6.467 ms，主要涉及视频读取、图像预处理、边界框绘制和结果整理。在本机测试条件下，完整处理速度全程高于 5 FPS。主要结果汇总于表IV。

TABLE IV
主要实验结果汇总

测试项目	结果
测试集规模	650 张，1107 个目标框
总体 mAP@0.5 / mAP@0.5:0.95	0.8386 / 0.5735
20 样本正确率	19/20，95%
平均推理耗时	10.087 ms
平均端到端速度	60.409 FPS
最低瞬时速度	34.733 FPS

VII. ROS2 通信与边缘部署设计

A. 检测节点与主题接口

检测程序封装为独立的 ROS2 节点。使用 USB 摄像头时，节点直接读取视频帧；接入已有相机驱动时，节点订阅 `/camera/image_raw`。每帧推理结束后发布两个主题：`/object_detection/result` 使用 JSON 字符串记录时间戳、帧号、FPS、类别编号、类别名称、置信度和边界框，`/object_detection/image` 则以 `sensor_msgs/Image` 传输带框图像。

数值结果与可视化图像分开发布，控制程序只接收 JSON 即可获取目标信息，调试程序则订阅带框画面。ROS2 允许多个订阅者同时接收同一主题 [9]。运行时可用 `ros2 topic echo` 检查消息字段，以 `ros2 topic hz` 统计接收频率，并在 `rqt_image_view` 中查看检测画面。

B. 边缘推理流程

部署模型采用训练生成的 `best.pt`。权重既可由 PyTorch 直接加载，也可导出为 ONNX，再通过 TensorRT

构建序列化引擎。TensorRT 根据目标 GPU 选择各层实现，运行时加载引擎完成推理 [10]。序列化引擎与操作系统、CPU 架构、GPU 型号及 TensorRT 版本相关，因而需在目标设备上生成，并固定输入尺寸与计算精度。

部署配置沿用训练时的类别顺序与 640 像素输入。采用 FP16 时保持置信度阈值和 NMS 参数不变。测速前先处理若干预热帧，以排除首次加载和内核初始化的影响，随后统计固定帧数的端到端 FPS。若需进一步降低计算量，可比较 640、512 和 416 三种输入尺寸；每次改变尺寸后重新执行 20 张样本测试，以检验速度提升对识别正确率的影响。

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[2] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “SSD: Single shot multibox detector,” in *Computer Vision – ECCV 2016*, 2016, pp. 21–37.

[3] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2980–2988.

[4] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal speed and accuracy of object detection,” *arXiv preprint arXiv:2004.10934*, 2020.

[5] G. Jocher and J. Qiu, “Ultralytics YOLO11,” 2024, version 11.0.0, accessed: 2026-09-03. [Online]. Available: https://github.com/ultralytics/ultralytics

[6] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Malloci, A. Kolesnikov, T. Duerig, and V. Ferrari, “The open images dataset v4: Unified image classification, object detection, and visual relationship detection at scale,” *International Journal of Computer Vision*, vol. 128, no. 7, pp. 1956–1981, 2020.

[7] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, “Microsoft COCO: Common objects in context,” in *Computer Vision – ECCV 2014*, 2014, pp. 740–755.

[8] Ultralytics, “Model validation with ultralytics YOLO,” https://docs.ultralytics.com/modes/val/, 2026, accessed: 2026-09-03.

[9] Open Robotics, “ROS 2 documentation: Topics,” https://docs.ros.org/en/ros2_documentation/kilted/Concepts/Basic/About-Topics.html, 2026, accessed: 2026-09-03.

[10] NVIDIA Corporation, “How TensorRT works,” https://docs.nvidia.com/deeplearning/tensorrt/latest/architecture/how-trt-works.html, 2026, accessed: 2026-09-03.